**Version** 2.0

January 2026

# OPERATING SYSTEMS PRACTICUM

## MODULE 2 - INTRODUCTION TO VIM EDITOR (TEXT MANIPULATION)

**Author**:

Ir. Denar Regata Akbi, S.Kom., M.Kom.

Putri Nayla Sabri

Muhammad Zamzam Baihaqi

**INFORMATICS LABORATORY**

**University of Muhammadiyah Malang**

## TABLE OF CONTENTS

# INTRODUCTION

## OBJECTIVES

1. Understand the fundamental concepts and functions of the Vim text editor in a Linux CLI environment.
2. Explain the role of Vim as a powerful, keyboard-driven tool for efficient text manipulation.
3. Identify the three core modes of Vim: Normal, Insert, and Command mode.
4. Distinguish between basic Vim commands for navigation, editing, saving, and searching.
5. Describe how to apply Vim commands to complete practical file editing tasks in lab assignments.

## MODULE TARGETS

1. Have a clear understanding of Vim's mode-based workflow and its advantages over GUI editors.
2. Recognize the relationship between terminal-based editing and system administration tasks in Linux.
3. Practice essential Vim operations: creating files, inserting text, deleting lines, copying/pasting, and replacing content.
4. Prepare an appropriate environment for practicing Vim commands using Rocky Linux in VirtualBox.
5. Build foundational skills for manipulating text files efficiently — crucial for completing Module 2 demo tasks.

## PREPARATION

1. Muslim students may recite the following prayer before starting the lesson

   رَضِيْتُ بِاللهِ رَبًّا وَبِالإِسْلَامِ دِيْنًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُوْلًا. رَبِّ زِدْنِيْ عِلْمًا وَارْزُقْنِيْ فَهْمًا

2. Laptop or personal computer
3. Rocky Linux installed via VirtualBox (or UTM)
4. Access to terminal (CLI) with sudo privileges
5. Strong motivation and commitment to learning keyboard-centric editing

## KEYWORDS

Vim Editor, Text Manipulation, Normal Mode, Insert Mode, Command Mode, vi Improved, CLI Editing, Search and Replace, :wq, :q!, vimtutor, Terminal Editor, Linux Text Editor

## 1. Introduction to Vim in the Rocky Linux Environment

### 1.1 Installing and verifying Vim (vim, vim-enhanced)

Vim is **not always pre-installed** on minimal installations of Rocky Linux. This section guides students to install, verify, and understand the different Vim packages available.

**A. Verify Vim if already installed**

To check if you already have vim installed, open your terminal in Rocky Linux and run:

```
praktikumso@localhost:~$ which vim
/usr/bin/vim
```

If your vim is already installed, it will have output like this, but if there is no output to be found that means you don't have vim yet and will need to install it.

You can also check for specific packages using dnf:

```
praktikumso@localhost:~$ dnf list installed | grep vim
vim-common.x86_64                              2:9.1.083-6.el10                @AppStream
vim-data.noarch                                2:9.1.083-6.el10                @anaconda
vim-enhanced.x86_64                            2:9.1.083-6.el10                @AppStream
vim-filesystem.noarch                          2:9.1.083-6.el10                @anaconda
vim-minimal.x86_64                             2:9.1.083-6.el10                @anaconda
```

This is the output if you already have vim installed. However, if there is no output, it means vim is not installed.

**B. Installing Vim via DNF Package Manager**

Run the following command as root or with sudo:

```
praktikumso@localhost:~$ sudo dnf install vim-minimal -y
```

**C. Verifying Installation and Version**

After installation, verify Vim's presence and version:

```
praktikumso@localhost:~$ vim --version
VIM - Vi IMproved 9.1 (2024 Jan 02, compiled Sep 10 2025 00:00:00)
Included patches: 1-83
Modified by <bugzilla@redhat.com>
Compiled by <bugzilla@redhat.com>
```

If you see version info, Vim is successfully installed.

**D. Understanding Vim Package Variants in Rocky Linux**

| Package Name | Description | Recommended for Students? |
|---|---|---|
| `vim-minimal` | Barebones Vim — no syntax highlighting, no mouse, no plugins | no |
| `vim-enhanced` | Full-featured Vim — includes syntax, folding, plugins, mouse, etc. | yes |
| `vim-X11` | GUI version (requires X11 desktop) | optional |

You should always aim for **vim-enhanced**. While `vim-minimal` is sufficient for emergency edits, it lacks the visual cues (syntax highlighting) that help prevent configuration errors in a production environment. If you type `vi` and the text is all one color, you are likely using the minimal version.

## 1.2 Vim operating modes

Vim's modal **editing system** is one of its most defining and initially challenging features, but it is also the source of its efficiency and precision. Unlike conventional text editors that remain in a single "typing" state, Vim separates interaction into **distinct modes to optimize different** tasks: **Normal mode** serves as the command center for navigation, deletion, copying, and macro execution; **Insert mode** allows direct text input like a standard editor; **Visual mode** enables intuitive selection of characters, lines, or blocks for operations like cut, copy, or indentation; and **Command-line mode** provides access to powerful file operations, search-and-replace, configuration settings, and built-in help.

**A. The Four Primary Modes**

| Mode | Purpose | How to Enter | How to Exit |
|---|---|---|---|
| Normal Mode | Navigation and command execution | Press Esc (default after startup) | N/A (base mode) |
| Insert Mode | Typing and editing text | Press i, a, o, I, A, O | Press Esc |
| Visual Mode | Selecting blocks of text | Press v (char), V (line), Ctrl+v (block) | Press Esc |
| Command-Line Mode | Executing commands (:w, :q, :s) | Press : (colon) from Normal Mode | Press Enter or Esc |

**B. Switching Between Modes – Practical Workflow**

Example workflow:

- Open file: `vim /etc/hosts`
- Press `i` → enter Insert Mode → type new entry
- Press `Esc` → back to Normal Mode
- Press `:w` → save file
- Press `:q` → quit Vim

Practice this cycle repeatedly until it becomes muscle memory.

**C. Common Mode-Specific Shortcuts**

In **Normal Mode**:

- `h`, `j`, `k`, `l` → left, down, up, right
- `w` / `b` → jump forward/backward by **word** (more efficient).
- `0` → go to start of line
- `$` → go to end of line
- `gg` → go to first line
- `G` → go to last line
- `/word` → search for a specific word. Press `n` to find the next occurrence.

In **Insert Mode**:

- Ctrl + o → execute one Normal Mode command then return to Insert Mode
- In Command-Line Mode:
- `:set nu` → show line numbers (crucial for debugging config files).
- `:w` → save
- `:q` → quit
- `:wq` or `:x` → save and quit
- `:q!` → quit without saving

**Tip**: Use :help anytime to access built-in documentation.

## EXERCISE 1

Open the `mode.txt` file with Vim, go to Insert Mode, type "Hello Vim", verify the output and then go back to Normal Mode and save the file.

## 1.3 Basic navigation in system configuration files

Vim is widely used by system administrators to edit critical configuration files. This section provides hands-on practice with real system files.

**A. Editing /etc/hosts – Adding a Local Host Entry**

To enter vim mode, use the vim command and type the file you want to edit, example:

```
praktikumso@localhost:~$ sudo vim /etc/host
```

After press enter you will be inside the file, and in this mode you can do editing for example click 'i' will trigger **insert mode** so you can add inside the file. Try adding the entry `127.0.0.1` mylocal.dev inside the file using insert mode.

```
127.0.0.1   mylocal.dev
~
-- INSERT --
```

After you finish adding the file using insert mode, press esc and then type `:wq` to save and quit from vim mode.

**B. Editing `/etc/fstab` – Mounting a Filesystem (Read-Only Practice)**

**Note:** Never modify `/etc/fstab` without understanding the consequences

Open file `fstab` using vim with read-only mode

```
praktikumso@localhost:~$ sudo vim -R /etc/fstab
```

You can navigate the file by pressing j for go down, k for go up ,gg for go to the first line, and G to go to the last line of the file.

You can search in this mode by pressing `'/'` and type what you want to search inside the file. For example:

```
#
# /etc/fstab
# Created by anaconda on Fri Jan  9 04:06:24 2026
#
# Accessible filesystems, by reference, are maintained under '/dev/disk/'.
# See man pages fstab(5), findfs(8), mount(8) and/or blkid(8) for more info.
#
# After editing this file, run 'systemctl daemon-reload' to update systemd
# units generated from this file.
#
UUID=669a220f-b8f4-4edc-a81b-56770940ad66 /                       xfs     defaults        0 0
UUID=af04e854-7fe1-49c5-8632-59af3aadd73d /boot                   xfs     defaults        0 0
UUID=15b9df6c-b0ac-4238-80ab-94042bf377f3 none                    swap    defaults        0 0
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
/UUID
```

You can also press `'n'` to jump to the next match. After you finish and want to quit this mode, just press Esc and type `:q`. This will quit without saving the change.

**C. Best Practices When Editing System Files**

| Practice | Reason |
|---|---|
| Always backup before edit | e.g., cp /etc/hosts /etc/hosts.bak |
| Use sudo when needed | Avoid permission denied errors |
| Test changes after save | e.g., ping mylocal.dev or mount -a |
| Use :set number | Display line numbers for easier reference |

**D. Quick Reference Table: Essential Commands for SysAdmin Tasks**

| Task | Command in Vim |
|---|---|
| Enable line numbers | `:set number` |
| Disable line numbers | `:set nonumber` |
| Search forward | `/keyword` + Enter |
| Search backward | `?keyword` + Enter |
| Jump to line 10 | `:10` + Enter |
| Save file | `:w` |
| Quit without saving | `:q!` |
| Save & quit | `:wq or :x` |

## 2. Text Manipulation for System Administration

Vim is far more than a text editor—it is a powerful tool for **system administration** in Linux environments. In Rocky Linux, critical tasks such as **editing configuration files** (`/etc/`) or **analyzing log files** (`/var/log/`) are routinely performed via the command line. Vim enables fast, safe, and efficient editing without requiring a graphical interface—making it **indispensable** for server management.

### 2.1 Efficient editing of logs and config files

**A. Why Edit Logs and Config Files in Vim?**

- Log files (e.g., `/var/log/messages`) help diagnose system or service errors.
- Configuration files (`/etc/httpd/conf/httpd.conf`) control how services behave.
- Many servers run without a GUI (headless), so CLI-based tools like Vim are essential.

Never edit active log files directly, only view them.

Always create a backup before modifying configuration files,example:

```
praktikumso@localhost:~$ sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak
```

**Practical Example: Editing SSH Configuration**

Goal: Change the default SSH port from 22 to 2222.

Open the configuration file:

```
praktikumso@localhost:~$ sudo vim /etc/ssh/sshd_config.bak
```

Press / and type Port, then press Enter to locate the line `#Port 22`.

Press i to enter Insert Mode, remove the #, and change 22 to 2222:

```
Port 2222
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::

#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_ecdsa_key
#HostKey /etc/ssh/ssh_host_ed25519_key

# Ciphers and keying
#RekeyLimit default none
-- INSERT --
```

Press Esc, then save and exit :

```
Port 2222
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::

#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_ecdsa_key
#HostKey /etc/ssh/ssh_host_ed25519_key

:wq
```

Test the configuration syntax before restarting:

```
praktikumso@localhost:~$ sudo sshd -t
```

If no errors appear, restart the SSH service:

```
praktikumso@localhost:~$ sudo systemctl restart sshd
```

**B. Viewing Large Log Files Efficiently**

Use read-only mode to safely inspect large logs without risking accidental writes:

```
praktikumso@localhost:~$ sudo vim -R /var/log/messages
[sudo] password for praktikumso:
```

Quick navigation commands:

- `G` → jump to end of file (newest logs are usually at the bottom)
- `gg` → jump to beginning
- `/error` → search for "error"
- `n` → next match, N → previous match

## EXERCISE 2

Edit the `/etc/hosts` file (use `sudo`) and add the line `127.0.0.1 demo.local` without changing any other contents.

## 2.2 Search and replace operations using basic patterns

Vim supports powerful search-and-replace operations using basic regular expressions—ideal for updating multiple configuration entries at once.

**A. Basic Search Syntax**

| Action | Command in Normal Mode |
|---|---|
| Search forward | `/keyword` |
| Search backward | `?keyword` |
| Repeat last search | `n` (next), `N` (prev) |

**Example:**

`/Listen` → find all occurrences of "Listen" in an Apache config file.

**B. Replace Commands (Substitution)**

General syntax:

```
:[range]s/pattern/replacement/[flags]
```

**Common Examples:**

| Task | Reason |
|---|---|
| Always backup before edit | e.g., cp /etc/hosts /etc/hosts.bak |
| Use sudo when needed | Avoid permission denied errors |
| Test changes after save | e.g., ping mylocal.dev or mount -a |
| Use :set number | Display line numbers for easier reference |

| Task | Command |
|---|---|
| Replace first match on current line | `:s/old/new/` |
| Replace all matches on current line | `:s/old/new/g` |
| Replace all matches in entire file | `:%s/old/new/g` |
| Replace with confirmation per change | `:%s/old/new/gc` |
| Case-insensitive replace | `:%s/OLD/new/gi` |

**Practical Example:**
Replace all http:// with https:// in a config file:

```
:%s/http:\/\//https:\/\//g
```

Explanation: The / character must be escaped with \ because it's used as the delimiter.
**Better alternative:** Use a different delimiter (e.g., #) for readability:

```
:%s#http://#https://#g
```

**C. Case Study: Mass Update Hostname References**
Suppose all config files reference localhost, and you want to update them to mylocalhost.
Open the hosts file:

```
praktikumso@localhost:~$ sudo vim /etc/hosts
```

Before:

```
# Loopback entries; do not change.
# For historical reasons, localhost precedes localhost.localdomain:
127.0.0.1   localhost localhost.localdomain localhost4 localhost4.localdomain4
::1         localhost localhost.localdomain localhost6 localhost6.localdomain6
# See hosts(5) for proper format and other examples:
# 192.168.1.10 foo.example.org foo
# 192.168.1.13 bar.example.org bar
```

run :

```
:%s/localhost/mylocalhost/g
```

After:

```
# Loopback entries; do not change.
# For historical reasons, mylocalhost precedes mylocalhost.localdomain:
127.0.0.1   mylocalhost mylocalhost.localdomain mylocalhost4 mylocalhost4.localdomain4
::1         mylocalhost mylocalhost.localdomain mylocalhost6 mylocalhost6.localdomain6
# See hosts(5) for proper format and other examples:
# 192.168.1.10 foo.example.org foo
# 192.168.1.13 bar.example.org bar
```

## EXERCISE 3

Create a `url.txt` file containing `http://site.com`, then replace all `http://` with `https://` using the global substitution command in Vim.

## 2.3 Multi-line editing and visual block mode for batch configuration tasks

When managing multiple services or servers, you often need to modify several lines simultaneously. Visual Block Mode allows column-based editing—a unique and powerful Vim feature.

**A. Entering Visual Block Mode**

- Press Ctrl + v → enter Visual Block Mode.
- Use arrow keys or `h`, `j`, `k`, `l` to select a rectangular block.
- Press I (capital i) to insert text at the start of each selected line.
- Type your text (e.g., # to comment out).
- Press Esc → the text appears on all selected lines.

**Practical Example: Comment Out Multiple Cron Jobs**

File: `/etc/crontab`

Goal: Temporarily disable several cron jobs.

1. Open the file:

   ```
   praktikumso@localhost:~$ sudo vim /etc/crontab
   ```

2. Move cursor to the first job line.
3. Press Ctrl + v, then j twice to select 3 lines.
4. Press I, type #, then press Esc.

Result:

```
#SHELL=/bin/bash
#PATH=/sbin:/bin:/usr/sbin:/usr/bin
#MAILTO=root
```

All lines are instantly commented!

**B. Advanced: Adding Indentation or Aligned Text**

You can also:

- Add indentation: select block → press > (indent right) or < (indent left)
- Insert aligned prefixes using substitution

Example (add 4 spaces to selected lines):

1. Select block with Ctrl+v
2. Press : → Vim shows :'<,'>
3. Type:

```
:'<,'>s/^/ /
```

This adds four spaces at the start of each selected line.

**Comparison: Line vs. Block Selection**

| Mode | Shortcut | Best For |
|---|---|---|
| Visual Line | V | Selecting full lines |
| Visual Character | v | Selecting text character by character |
| Visual Block | Ctrl+v | Column editing, batch comments, alignment |

## EXERCISE 4

Create three lines of random text, then use Visual Block Mode (`Ctrl+v`) to add a `#` sign at the beginning of all three lines at once.

## 3. Customizing Vim for SysAdmin Productivity

Vim is highly configurable, allowing system administrators to **tailor its behavior** to match their workflow. Unlike basic text editors, Vim **supports persistent customization** through a configuration file named .vimrc, located in the user's home directory. By optimizing this file, users can significantly **improve editing speed**, **reduce errors**, and **enhance readability** especially when working with complex system files or scripts.

This section introduces essential customizations that every student should implement early in their Linux journey to build efficient, professional-grade CLI habits.

### 3.1 Basic .vimrc setup

**A. What Is .vimrc?**

The .vimrc file (short for Vi IMproved Runtime Configuration) is a plain-text file that Vim reads at startup. It contains user-defined settings and commands that override Vim's default behavior. If the file does not exist, Vim runs with factory defaults—often lacking features critical for system administration.

Note: The dot (.) at the beginning makes it a hidden file in Linux. Use ls -la ~ to view it.

**B. Creating and Editing .vimrc**

Open or create the file:

```
praktikumso@localhost:~$ vim ~/.vimrc
```

Add the following essential lines:

```
" Enable syntax highlighting
syntax on

" Show line numbers
set number

" Enable auto-indentation
set autoindent
set smartindent
set tabstop=4
set shiftwidth=4
set expandtab
```

Save and exit:

```
set tabstop=4
set shiftwidth=4
set expandtab

:wq
```

Explanation of Key Settings:

- syntax on: Enables color-coded highlighting based on file type .
- set number: Displays line numbers—critical for referencing logs or debugging.
- autoindent & smartindent: Automatically align new lines with the previous one.
- tabstop=4 & expandtab: Converts tabs to 4 spaces (standard in most Linux configs).

**C. Verifying the Configuration**

Open any file (e.g., a shell script):

```
praktikumso@localhost:~$ vim test.sh
```

Type:

```
1 #!/bin/bash
2 if [ true ]; then
3     echo "Hello"
4 fi
```

You should see:

- Line numbers on the left
- Colored keywords (if, then, echo)
- Proper indentation after pressing Enter

If not, check for typos in .vimrc.

### EXERCISE 5

Create a `~/.vimrc_exercise` file and add four lines: `syntax on`, `set number`, `set tabstop=4`, and `set expandtab`.

## 3.2 Shell integration

One of Vim's most powerful features is its ability to interact with the shell without exiting the editor. This is invaluable when you need to check system status, list files, or test a command while editing a configuration.

**A. Basic Shell Command Execution**

From Normal Mode, press `:` and type `!` followed by a shell command:

```
:!ls -l
```

This runs ls `-l` in the terminal and displays the output. Press Enter to return to Vim.

**Common Examples:**

| Command in Vim | Purpose |
|---|---|
| `:!pwd` | Show current working directory |
| `:!whoami` | Display current user |
| `:!systemctl status sshd` | Check SSH service status |
| `:!date` | Show current date/time |

**B. Inserting Command Output into File**

You can insert the output of a shell command directly into your document:

```
:r !date
```

This inserts the current date at the cursor position.

Example use case:

- Add a timestamp comment to a log file:

```
1 # Last updated:
```

Then run:

```
:r !date
```

Result:

```
5 Thu Jan 22 06:21:22 PM WIB 2026
```

**C. Editing Files Based on Command Output**

You can even open files listed by a command:

```
:e /etc/hosts
```

Or combine with shell expansion:

```
:e ~/.vimrc
```

Caution: Commands run with the same privileges as your user. Use sudo outside Vim if root access is needed.

## 3.3 Efficiency tips

Advanced Vim users leverage built-in tools to manage large or multiple files efficiently—skills especially useful when editing long config files like `/etc/httpd/conf/httpd.conf` or multi-section scripts.

**D. Using Bookmarks (Marks)**

Vim allows you to set bookmarks (called marks) to jump quickly between locations.

- Set a mark named a at current line:
  **INSIDE VIM MODE PRESS M + A (on selected row and column)**
- Jump back to mark a:
  **PRESS ‘ + A on keyboard (inside same vim that already marked)**
- Jump to line of mark a (without column):
  **PRESS ` + a on keyboard (inside same vim that already marked)**

Use Case:

While editing `/etc/fstab`, mark the swap line with ms, scroll to network mounts, then return instantly with 's.

Note: Lowercase marks (a–z) are local to the file; uppercase (A–Z) are global.

**E. Code Folding for Large Files**

Folding collapses sections of text, code, or config blocks, making navigation easier.

Enable manual folding in .vimrc:

```
1 set foldmethod=manual
```

Then:

- Select lines with V (Visual Line mode)
- Press `zf` to create a fold
- Press `zo` to open a fold, `zc` to close it

Alternatively, use syntax-based folding (for scripts):

```
1 set foldmethod=syntax
```

**F. Navigating Between Multiple Files**

Vim supports multiple buffers (open files in memory).

| Command | Action |
|---|---|
| `:e` filename | Open another file in the same window |
| `:ls` | List all open buffers |
| `:bnext or :bn` | Switch to next buffer |
| `:bprev or :bp` | Switch to previous buffer |
| `:b 2` | Switch directly to buffer #2 |
| `:bd` | Close (delete) current buffer |

Pro Tip: Combine with .vimrc setting:

```
1 set hidden
```

This allows switching buffers without saving—essential for quick comparisons.

**G. Practical Workflow Example**

Scenario: Compare /etc/ssh/sshd_config and a backup /etc/ssh/sshd_config.bak.

1. Open first file:

```
vim /etc/ssh/sshd_config
```

2. Inside Vim, open second file:

```
:e /etc/ssh/sshd_config.bak
```

3. Switch between them:

```
:bp    " go back to original
:bn    " go to backup
```

4. Use marks to remember key lines (e.g., Port setting).

## PRACTICAL APPLICATIONS

### PRACTICE 1

- Create a folder named **LatihanVim**.
- Enter that folder, then create a file **config.txt** containing:

```
1 url = http://test.com
2 port = 80
3 debug = true
```

- Open **config.txt** with Vim, then replace `http://test.com` with `https://prod.com` using the search and replace command (`:%s`).
- Use **Visual Block Mode** (`Ctrl+v`) to add a `#` character at the beginning of the line `debug = true`.
- Create a file **~/.vimrc_praktik1** and fill it with:

```
1 syntax on
2 set number
```

- Return to **config.txt**, then insert the current date at the end of the file using the command `:r !date`.

### PRACTICE 2

- Create a folder named **LatihanVim2**.
- Enter that folder, then create a file **settings.conf** containing:

```
server.port=8080
server.host=localhost
app.name=MyApplication
```

- Open the directory **/etc/hosts** using **sudo Vim**, then replace `localhost` with `myapp.local` using the search and replace command (`:%s`).
- Mark the first line with mark **h** (`mh`), then move to the end of the file and return to that mark using `'h`.
- Create a file **~/.vimrc_praktik2** and add:

```
set number
set tabstop=4
set autoindent
```

```
syntax on
```

- Run the shell command `:!whoami` from within Vim, then save its output to the end of the file using `:r !whoami`.

---

## DEMO – PROJECT CASE STUDY

**The Situation**

You are a junior system administrator at the UMM Informatics Laboratory. You have been assigned to maintain a Rocky Linux server that hosts a psychology portal web application. The development team has provided an outdated configuration file (`app.conf`) that is currently insecure and poorly documented.

**The Problem**

The existing configuration file has several critical issues:

- Security Risk: It uses insecure `http://` URLs.
- Debugging Active: The debug settings are still enabled, which is dangerous for a production environment.
- No Logging Info: The file does not record when the last update occurred.
- Unstandardized Format: It lacks comments and professional formatting.

**The Mission**

Your mission is to standardize and secure the `app.conf` file using only Vim. You must demonstrate proficiency in switching modes, performing bulk edits, and integrating shell commands for validation.

## DEMO TASKS (Student Must Perform Live in Terminal)

1. **Set Up Your Vim Environment**
   - Create a `~/.vimrc_Demo` file containing:

```
1 syntax on
2 set number
3 set tabstop = 4
4 set expandtab
```

   - Verify these settings take effect by opening any file in Vim.
2. **Populate app.conf with Required Content**
   - Create and open `/home/praktikumso/config/app.conf` using Vim.
   - Type the following content manually:

```
# PRODUCTION CONFIG - UPDATED JAN 2026

server_url = http://api.newcampus.ac.id
admin_panel = http://admin.newcampus.ac.id
debug = false
log_level = info
data_dir = /var/log/app
```

3. **Update URLs in Bulk**
   - Replace all occurrences of `http://` with `https://` across the entire file using a global substitution command.
4. **Standardize Format Using Visual Block Mode**
   - Use **Visual Block Mode** (`Ctrl+v`) to add a `#` at the beginning of these lines:

```
debug = false
log_level = info
```

   (This simulates disabling debug mode for production.)
5. **Integrate Shell Commands for Verification**
   - From within Vim:
   - Insert the current date into the file using `:r !date`,
   - Check if the directory `/var/log/app` exists using `:!ls -ld /var/log/app`.
6. **Save and Exit Properly**
   - Save your changes and exit Vim using the correct command (`:wq`).

---

## Demo Rules

1. **No Module or Notes Access**

Students **must not** open the module PDF, personal notes, or online documentation during the demo.

2. **Vim-Only Editing**

Only `vim` may be used. Editors like `nano`, `gedit`, or VS Code are **not permitted**.

3. **Demonstrate Core Vim Skills**

Students must clearly show understanding of:

- Switching between modes (Normal, Insert, Command-line),
- Using `:%s/pattern/replacement/g` for global search-and-replace,
- Applying **Visual Block Mode** (`Ctrl+v`) for batch commenting,
- Executing shell commands from Vim (`:!`, `:r !`),
- Configuring basic settings via `.vimrc`.

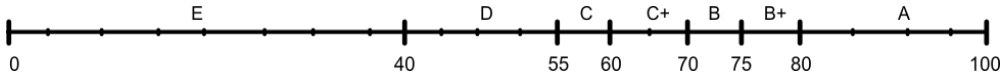
4. **Final Verification**
After completing the tasks, students must display the final file content using `cat` or `less` to confirm all changes were applied correctly.

5. **Follow Live Instructions**
All actions must be performed interactively following the instructor's or assistant's directions.

## ASSESSMENT RUBRIC

| Assessment Aspect | Points (Total 100%) |
|---|---|
| **EXERCISE** | **Total 5%** |
| Exercise 1 | 1% |
| Exercise 2 | 1% |
| Exercise 3 | 1% |
| Exercise 4 | 1% |
| Exercise 5 | 1% |
| **PRACTICE** | **Total 20%** |
| Practice 1 (Verbal assessment) | 5% |
| Practice 2 (Verbal assessment) | 5% |
| Practice Completion (Live verification) | 10% |
| **DEMO** | **Total 75%** |
| Case Study Completion | 35% |
| Understanding Transition Modes and Save/Exit | 15% |
| Understanding Regex Escaping | 10% |
| Understanding Visual Block Logic | 10% |
| Understanding Shell Integration | 5% |

## ASSESSMENT SCALE




**A = (81 - 100) → Sepuh**

**B+ = (75 - 80) → Very Good**

**B = (70 - 74) → Good**

**C+ = (60 - 69) → Fairly Good**

**C = (55 - 59) → Fair**

**D = (41 - 54) → Poor**

**E = (0 - 40) → Bro really...**